Branchmap: An infrastructure layer for multiverse finance

Jonathan Paprocki, Samuel Henriquez
jon@branchpoint.xyz, sam@branchpoint.xyz

Draft Version 2
July 24, 2026

Abstract

We introduce Branchmap, an EVM smart contract suite implementing the notion of 'multiverse map' from multiverse finance. It is a semantic namespace for events, conditional claims on assets, and integration layer for defi protocols to make them 'multiverse-aware'. We introduce a mathematical framework for multiverse finance, putting it in the context of Arrow-Debreu complete markets. We then explain how Branchmap realizes this mathematical framework, and then explore some applications, such as impact markets, how options are a special case of multiverse finance, and new financial instruments only made possible via this framework. We also discuss the mathematics of a multiverse portfolio.

Keywords: multiverse finance; prediction markets; conditional tokens; namespaces; EVM; defi; event identity; impact markets; branch cash; conditional assets.

1. Introduction

1.1 What is Branchmap?

In this paper we introduce concepts beginning to be known as multiverse finance, as well as Branchmap, an EVM blockchain semantic namespace designed to enable multiverse finance by wiring up event venues such as prediction markets and decentralized finance (defi) protocols such as decentralized exchanges, lending protocols, tokenized trading strategies, etc, into one substrate. Branchmap and its associated SDKs will enable on-chain finance developers and institutions to deploy a variety of novel financial instruments, including but not limited to: assets conditioned on the outcomes of one or more events, smart contracts whose behavior is contingent on event outcomes, lending and borrowing contingent on events, contingent perpetual futures, option-like multiverse "collars", contingent predictions markets, and innumerable other exotic instruments that were previously impossible. Branchmap provides the plumbing that allows existing defi protocol developers to make their protocols 'multiverse-aware', such that they can create alternate versions of their protocols where users are able to interact with that protocol as though an event outcome has already occurred. In addition, the onchain semantic nature of the Branchmap namespace allows for discoverability that makes it easy for humans and agents to determine which events have liquidity for which assets, which protocols serve given events, and how they are resolved, all in one place.

The core primitive of Branchmap is a namespace of events and possible outcomes. In general, these events are sourced from onchain prediction markets, as prediction markets binary outcome contracts function as "cash" in a given universe. That is to say, given an event e and outcome o, if you possess a binary outcome contract on a prediction market that is exchangeable for \$1 when e resolves as o, this is the same thing as having \$1 in the universe where e has resolved as o. Thus, prediction market binary outcome contracts serve as the numeraire in a given universe.

Each event e gets its own entry in the Branchmap namespace. Inside of this entry is listed every cryptographic asset (such as ERC-20's and ERC-721's), every DEX, every lending protocol, every perpetual future, etc. that form a coherent 'financial universe' that can be interacted with. Thus the namespace creates discoverability for events and every smart contract that is contingent on outcomes of that event.

1.2 Directional vs. non-directional traders

There are two primary modes by which a multiverse finance trader could interact with a given event: directional and non-directional.

A user who believes that e will impact their assets and wishes to position their portfolio ahead of time such that they have already made the trades that they would like to make under every possible outcome of e is working in the *non-directional* mode. Unlike in a prediction market, they are not making any sort of bet as to whether a given outcome occurs. They are depositing their assets into the Branchmap smart contract, receiving a copy of that asset conditioned on every possible outcome in the event, make trades and other financial decisions within each possible outcome, and then when the event does resolve and the outcome is finalized, the assets conditioned on that particular outcome are made 'real' once again.

A user who believes that a given asset under a given outcome, say BTC if e resolves as o, is underpriced, and would like to therefore buy BTC specifically in that universe, is said to be working in the *directional* mode. This type of

DRAFT

user has much more in common with prediction market traders than non-directional traders - they are stacking two bets: that a given outcome will occur, and also that a particular asset in the universe where we assume that that outcome has already occurred, is underpriced.

Our hypothesis is that non-directional traders will be much more common than directional traders, and thus that the size of the market for multiverse finance is a large multiple of prediction markets. Branchmap and multiverse finance in general enables both, but the design of Branchmap is opinionated and oriented around non-directional traders.

We strongly emphasize the following point that distinguishes what it means for a trader to be directional or non-directional in the multiverse finance sense: the kind of knowledge needed to make these decisions well is often very different as well as much more common than the type of knowledge needed to determine *whether* a particular will occur. Consider the Digital Asset Market Clarity Act [citation], also known as the CLARITY Act, at time of writing currently under consideration in the United State Congress that deeply impacts digital asset regulation. Being able to ascertain whether the current price of a binary outcome contract on a prediction market for the CLARITY Act passing is incorrect, and therefore have an edge from which the user can profit, requires deep knowledge of Congressional structure, the individual legislators writing the legislation, the negotiation process, various political tailwinds, and innumerable others factors. There are probably very few people who truly have an edge on this question. On the other hand, the actual effect of the CLARITY Act passing on various asset prices, requires much less specialized knowledge. Most traders believe that the passage of the CLARITY Act will have a positive impact on cryptocurrency prices, and that it failing to pass will have a negative impact. In a multiverse-finance enabled on-chain financial system, they will be able to position their assets according to each possible outcome accordingly without ever forming an opinion on whether or not the CLARITY Act will pass.

1.3 Why a namespace?

A namespace is valuable when it is is the caking agent for many related objects which can be grouped by name that need mutual discoverability. Multiverse finance begs for this because there are numerous event venues, assets, oracles, claims, resolution rules, token wrappers, pools, lending markets, and composite instruments that all need mutual compatibility with one another in order to form a coherent system.

Another reason we created the Branchmap namespace is to give event venues and defi protocol operators ownership over their own domains of the namespace. An event venue, for instance, will be able to claim a top-level domain under which they will be able to control how events are listed and resolved, their metadata, and create additional allowances within the namespace specific to their venue that the default template for event venues does not provide. A DEX operator, on the other hand, will be able to claim a subdomain under a given event and list the contracts for all assets available for trade on their DEX contingent on that event.

The initial pilot program of Branchmap, however, will be more centrally controlled by Branchpoint Labs, the corporate entity developing Branchmap. This is to ensure that we can react quickly to any issues that may arise, as decentralized processes move slowly. As we discuss with ecosystem partners over time, we will refine what affordances the namespace needs to have to successful decentralize over time, leading towards the ultimate goal of making Branchmap into the credibly neutral 'namespace for all events'.

1.4 Prior art

Here we discuss various related projects that have inspired the design and architecture of Branchmap.

1.4.1 Gnosis conditional token framework

The Gnosis Conditional Token Framework [citation], widely used by prediction markets going as far back as Omen, and today by PM's such as Polymarket and Limitless, provides event-conditioned assets, and those familiar with Gnosis CTF and the commutative settlement algebra for assets conditioned on multiple events is replicated within Branchmap. That is, within both Gnosis CTF and Branchmap, you can condition an asset on more than one event, and the order in which you condition on the events does not matter.

The primary distinction between Gnosis CTF tokens and Branchmap conditioned ERC-20's is that Gnosis tokens utilize an opaque identifier (the address of the oracle that resolves the event). If someone hands you a Gnosis CTF token with no additional context, there is no way to ascertain which event(s) and which outcome(s) it is conditioned on. A Branchmap conditioned ERC-20, on the other hand, has a semantic identifier: a slug for the name of the event and outcome, which is a lookup key for the Branchmap namespace, which then gives you the event, its full resolution text, the oracle which resolves it, the event venue it was sourced from, and the addresses of every asset, lending protocol, etc. that is also contingent on this event.

1.4.2 Hypermap

The EVM Branchmap smart contract suite is a fork of Hypermap [citation], an onchain general purpose EVM namespace developed by Hyperware that functions as programmable substrate for identity, ownership, delegation, metadata, and much more. We have stripped Hypermap of several of its features and rewritten several others to make it suitable for the purposes of Branchmap. No familiarity with Hypermap is required to understand this paper, and

DRAFT

BRANCHMAP is overall a much simpler architecture. We discuss the precise structure of BRANCHMAP in section 3.3.

1.4.3 Ethereum Name Service

ENS demonstrates that human-meaningful names can coordinate wallets, contracts, metadata, and identity across applications. BRANCHMAP applies the namespace idea to financial states of the world rather than human-readable account names.

1.4.4 Chainlink and other oracle namespaces

Chainlink established an on-chain oracle data directory [citation] using the ENS namespace `data.eth`. By giving oracles a human-readable naming system, developers and decentralized application are easily able to locate specific oracle feed addresses without hardcoding raw hexadecimal addresses. BRANCHMAP is designed to be compatible with ENS oracle namespaces, allowing the use of an oracle name rather than an address to resolve a given event.

1.4.5 Prediction market venues

Prediction market venues create liquid markets around claims about future events. BRANCHMAP provides a shared semantic layer through which multiple venues can publish, claim, compare, and compose markets, and users can determine which event venue and event a given asset is conditioned on via an on-chain namespace lookup.

1.4.6 Defi composability

Existing defi protocols compose around primarily ERC-20 tokens, pools, vaults, and debt positions. Multiverse finance extends this composability into conditional worlds: financial systems whose assets, liabilities, and contracts are scoped to particular future states.

1.5 Audience

This paper is primarily written for four audiences:

1. **Technical builders**, who need a precise model of events, propositions, partitions, predicates, conditional tokens, and wrappers.
2. **Traders**, who need to understand how to interact with multiverse finance as well as the basic mathematical concepts of a multiverse finance portfolio.
3. **Ecosystem partners**, including DEXes, prediction markets, event venues, oracle networks, and lending protocols that may integrate with BRANCHMAP.
4. **Investors**, who need to understand why multiverse finance may become a new category of financial infrastructure.
5. **Economists and mathematicians**, who would like to understand some of the mathematics of multiverse finance.

1.6 Scope

This paper covers the basic concepts of multiverse finance, elementary mathematical concepts of multiverse finance and a model for progressive refinement of the Arrow-Debreu state space, new types of financial instruments which could only have been made in a multiverse finance context, the design and architecture of BRANCHMAP in its first iteration as an EVM smart contract suite as it currently exists at time of writing, and our future plans for a cross-chain, virtual machine independent BRANCHMAP to enable onchain multiverse finance everywhere.

2. Multiverse Finance

In this section we introduce multiverse finance from a mathematical perspective and relate it to Arrow–Debreu securities and complete markets.

Throughout, let $(\Omega, \mathcal{F})$ be a measurable space of possible future world-states. A point $\omega \in \Omega$ represents one complete possible realization of the future.

2.1 Pricing convention

Fix a maturity or resolution time T and a settlement asset in which all payoffs are denominated.

A *terminal payoff* is a measurable function

$$Y_T : \Omega \longrightarrow \mathbb{R},$$

where $Y_T(\omega)$ is the number of units of the settlement asset delivered at time T if the realized world-state is $\omega \in \Omega$.

We assume that terminal payoffs are integrable under the state-price measure ν introduced below:

$$\int_\Omega |Y_T(\omega)|\ \nu(d\omega) < \infty.$$

This condition ensures that every payoff considered has a finite present value. Bounded payoffs, including binary prediction-market claims, automatically satisfy this condition. We restrict attention to option payoffs that are integrable.

Let ν be a positive finite measure on $(\Omega, \mathcal{F})$, called the *state-price measure*. For a measurable set $V \in \mathcal{F}$, the quantity $\nu(V)$, called the *state price* of V, is the present price of a claim that pays one unit of the settlement asset at time T if V occurs and zero otherwise.

The present value $V_0(Y)$ of a terminal payoff Y_T is therefore

$$V_0(Y) := \int_\Omega Y_T(\omega)\, \nu(d\omega). \tag{1}$$

The *discount factor* from time T to the present is defined by

$$D_{0T} := \nu(\Omega) = V_0(\mathbf{1}_\Omega),$$

where $\mathbf{1}_\Omega$ is the constant-one indicator function on Ω. Thus D_{0T} is the present price of a claim that pays one unit of the settlement asset at time T in every possible world-state.

DRAFT

Assuming

$$0 < D_{0T} < \infty,$$

we define the *normalized pricing measure* $\mathbb{Q}$ by

$$\mathbb{Q}(A) := \frac{\nu(A)}{D_{0T}}, \qquad A \in \mathcal{F}.$$

Since

$$\mathbb{Q}(\Omega) = \frac{\nu(\Omega)}{D_{0T}} = 1,$$

the measure $\mathbb{Q}$ is a probability measure. The pricing equation (1) may then be written

$$V_0(Y) = D_{0T} \int_\Omega Y_T(\omega)\, \mathbb{Q}(d\omega) = D_{0T}\, \mathbb{E}^{\mathbb{Q}}[Y_T], \qquad (2)$$

where $\mathbb{E}^{\mathbb{Q}}[Y_T]$ is the expectation value of Y_T with respect to $\mathbb{Q}$.

The measure $\mathbb{Q}$ is a market-implied pricing measure. It need not coincide with the physical probability distribution governing real-world frequencies or with any particular trader's subjective beliefs.

2.1.1 Finite partitions

In the real world, we are not dealing with infinite spaces of world states. Computers can only handle a finite number of them. Thus for practical purposes, we define the following notation.

Let $\pi = \{V_1, \ldots, V_n\}$ be a finite partition of Ω. Then we define

$$\begin{aligned}
\psi_i &:= \nu(V_i), && \text{state price of } V_i,\\
q_i &:= \mathbb{Q}(V_i) = \frac{\psi_i}{D_{0T}}, && \text{pricing probability of } V_i,\\
m_Y(V_i) &:= \mathbb{E}^{\mathbb{Q}}[Y_T \mid V_i], && \text{in-verse value of } Y,\\
C_Y(V_i) &:= V_0(Y_T \mathbf{1}_{V_i}), && \text{price of conditional } Y \text{ in } V.
\end{aligned}$$

These satisfy the *branch price factorization equation*, one of the most important equations of multiverse finance:

$$C_Y(V_i) = \psi_i m_Y(V_i) = D_{0T} q_i\, m_Y(V_i) \qquad (3)$$

for all $V_i \in \pi$, and

$$V_0(Y) = \sum_{i=1}^{n} C_Y(V_i) = \sum_{i=1}^{n} \psi_i m_Y(V_i).$$

In the context of Branchmap, we call the V_i *verses*, as in, the universes where the asset lives.

2.2 Arrow-Debreu theory

We view multiverse finance as a pragmatic attempt at utilizing existing prediction market infrastructure to extend the space of possible financial instruments towards the Arrow-Debreu model for complete markets [citation]. In this section we explain how multiverse finance fits into the model of Arrow-Debreu complete markets.

In the course of developing multiverse finance, we must thread the needle between adding too much complexity to the economy and not enough, and of what type. Multiverse finance provides a UX and opinionated take on the many different "coordinate systems" one could use to define products in the space of *Arrow securities*, essentially 'claims on assets contingent on some future event', defined below. New types of instruments create more avenues for risk-transfer. The more the economy grows in complexity, in terms of number and quantity of resources and resource transformation processes, the more we need new instruments to handle new types of risk. Not having enough instrumental complexity creates a market that is blind to its own actions, resulting in poorer outcomes. Having too much instrumental complexity results in a market in which actors cannot make rational decisions, resulting in poorer outcomes. Thus great care must be taken when creating new instruments. However, given the dramatically increased capacity to handle new instrumental complexity enabled by AI, we believe that now is the time in which bigger steps towards complete markets can be made.

We believe that existing risk-transfer mechanisms are too constrained to handle the exponentially growing amount of complexity in resource and resource transformations.

In simplified terms, Arrow-Debreu theory implies that the current price of an asset is a probabilistically-weighted average of the price of a claim on that asset in all future states of the world.

2.2.1 Arrow securities

Let $\pi = \{V_1, \ldots, V_N\}$ be a finite partition of Ω at some chosen terminal time T, and fix some settlement asset.

For each $V_i \in \pi$, define the associated *Arrow security* $B_i : \Omega \to \{0, 1\}$ to be the terminal payoff

$$B_i(\omega) := \mathbf{1}_{V_i}(\omega) = \begin{cases} 1, & \omega \in V_i, \\ 0, & \omega \notin V_i. \end{cases}$$

That is, B_i pays one unit of the settlement asset at time T if V_i occurs, and zero otherwise. Its present price is

$$V_0(B_i) = \nu(V_i) = \psi_i.$$

Prediction market contracts are Arrow securities. Here, B_i is the outcome token corresponding to verse V_i, and the settlement asset is \$1. For example, consider $\pi = \{V_{\mathsf{YES}}, V_{\mathsf{NO}}\}$. Then B_{YES} is what is commonly referred to as a YES contract or token, and B_{NO} is a NO contract or token, and $V_0(B_{\mathsf{YES}}) = \nu(V_{\mathsf{YES}})$ is the state price of V_{YES}. Often in the context of prediction markets, $D_{0T} = 1$, meaning that idle dollars are not collecting the risk-free rate. In that case, $\mathbb{Q}(V_{\mathsf{YES}}) = \nu(V_{\mathsf{YES}})$, the market-implied probability of V_{YES}, and this obtains the common understanding that the prices of prediction market contracts are the market-implied probability of an event occurring [citation].

In the context of Branchmap, we often call Arrow securities that resolve to a dollar in a given verse *branch*

DRAFT

cash, because you can trade the security with other assets conditional on the same verse as if it were a dollar. This is one way in which BRANCHMAP utilizes prediction markets as infrastructure - they provide the event definitions and resolvers, branch cash, and market-implied probabilities, that aid in defining other assets.

2.2.2 Conditional claims and branch prices

Let $Y_T : \Omega \to \mathbb{R}$ be an integrable terminal payoff, and let

$$\pi = \{V_1, \dots, V_n\}$$

be a finite measurable partition of Ω.

The claim on Y contingent on verse V_i, written $Y \mid V_i$, is the terminal payoff

$$(Y \mid V_i)_T := Y_T \mathbf{1}_{V_i}. \tag{4}$$

Its present price is the branch price

$$\begin{aligned} C_Y(V_i) &:= V_0(Y_T \mathbf{1}_{V_i}) \\ &= \int_{V_i} Y_T(\omega)\, \nu(d\omega). \end{aligned} \tag{5}$$

Since the verses form a partition,

$$Y_T = \sum_{i=1}^{n} Y_T \mathbf{1}_{V_i}.$$

Linearity of the pricing functional therefore gives

$$V_0(Y) = \sum_{i=1}^{n} C_Y(V_i).$$

Thus an ordinary asset is the aggregate of its verse-contingent components. At the level of terminal payoffs, we may write

$$Y \longleftrightarrow \big(Y \mid V_1, \dots, Y \mid V_n\big).$$

A core function of BRANCHMAP is support for this identity, in both directions: splitting and recombination. You may think of this as splitting, say, an ERC-20 token into n ERC-20 tokens, each one a conditional claim on an asset in a verse, and then recombining the n ERC-20 tokens into the base ERC-20. More on this in section 3.3.8.

The branch price already incorporates both the state price of the verse and the value of the asset within that verse. In the notation of Section 2.1,

$$C_Y(V_i) = \psi_i\, m_Y(V_i) = D_{0T} q_i\, m_Y(V_i).$$

Whenever $\psi_i > 0$, the market-implied value of Y inside the verse is

$$m_Y(V_i) = \frac{C_Y(V_i)}{\psi_i}.$$

This identity separates the two markets used by multiverse finance: prediction markets reveal the state price ψ_i, while markets for conditional assets reveal the branch price $C_Y(V_i)$. Their ratio reveals the market-implied value of the asset conditional on the verse occurring.

For branch cash $B_i = \mathbf{1}_{V_i}$, the in-verse payoff is one, so

$$C_{B_i}(V_i) = \psi_i.$$

Branch cash is therefore the elementary verse-contingent claim against which other assets in the same verse may be priced.

2.2.3 Finite claim spaces

For the remainder of this section, the world-state space $(\Omega, \mathcal{F})$ may be arbitrary, but we restrict attention to finite measurable partitions of Ω. This reflects the fact that any concrete market or application can distinguish only finitely many terminal states, even when the underlying world-state space is much larger. Everything here has an infinite analog that the reader is encouraged to derive according to their level of interest, but since BRANCHMAP is ultimately a finite product, the additional complexity surrounding e.g. topology is not worth the additional exposition required.

In this subsection, we show that every finite measureable partition defines a space of state-contingent claims.

Let

$$\mathrm{Part}_{\mathrm{fin}}(\Omega)$$

denote the partially ordered set of finite measurable partitions of Ω. We write

$$\pi' \succeq \pi$$

when π' refines π, meaning that every cell of π' is contained in a cell of π.

For a finite partition

$$\pi = \{V_1, \dots, V_n\},$$

define the claim space

$$\mathcal{C}_\pi := \left\{ \sum_{i=1}^{n} y_i \mathbf{1}_{V_i} : y_i \in \mathbb{R} \right\}.$$

These are precisely the terminal payoffs whose value depends only on which verse in π occurs. The correspondence

$$\sum_{i=1}^{n} y_i \mathbf{1}_{V_i} \longleftrightarrow (y_1, \dots, y_n)$$

gives an isomorphism $\mathcal{C}_\pi \cong \mathbb{R}^n$.

The coordinates y_i are verse-contingent payoffs, not prices. The state-price measure induces a linear pricing functional $\Lambda_\pi : \mathcal{C}_\pi \to \mathbb{R}$ given by

$$\Lambda_\pi \left(\sum_{i=1}^{n} y_i \mathbf{1}_{V_i} \right) := \sum_{i=1}^{n} \psi_i y_i. \tag{6}$$

A market is *complete relative to* π if every payoff in $\mathcal{C}_\pi$ can be traded or replicated. Completeness therefore depends on both the mathematical claim space and the set of financial instruments actually available.

DRAFT

If $\pi' \succeq \pi$, every π-measurable claim is also π'-measurable, giving a natural inclusion

$$\mathcal{C}_\pi \hookrightarrow \mathcal{C}_{\pi'}.$$

This refinement enlarges the space of distinctions on which payoffs may depend, and any coarser claim is decomposed into a sum of finer claims.

Prices must remain coherent under refinement. If $\pi' \succeq \pi$ and $V \in \pi$, then V is the disjoint union of the finer cells of π' contained within it:

$$V = \bigsqcup_{\substack{W \in \pi' \\ W \subseteq V}} W.$$

Since the state-price measure is additive,

$$\nu(V) = \sum_{\substack{W \in \pi' \\ W \subseteq V}} \nu(W). \tag{7}$$

Thus, the state price of a coarse verse equals the sum of the state prices of all finer verses into which it is subdivided.

The same additivity holds for conditional claims:

$$C_Y(V) = \sum_{\substack{W \in \pi' \\ W \subseteq V}} C_Y(W). \tag{8}$$

Thus refining a verse decomposes its branch price into the branch prices of its finer subverses, while coarsening adds those prices back together.

2.2.4 The limiting claim space

As the partition is refined, the claim spaces form a directed system

$$\mathcal{C}_\pi \hookrightarrow \mathcal{C}_{\pi'}.$$

A *simple* measurable payoff is a measurable payoff that takes only finitely many values. The space of simple measurable payoffs is the union of claim spaces over all finite measurable partitions:

$$\bigcup_{\pi \in \mathrm{Part}_{\mathrm{fin}}(\Omega)} \mathcal{C}_\pi.$$

Let $L^1(\nu) := L^1(\Omega, \mathcal{F}, \nu)$ denote the space of measurable terminal payoffs $Y_T : \Omega \to \mathbb{R}$ satisfying

$$\int_\Omega |Y_T(\omega)|\, \nu(d\omega) < \infty.$$

Because simple functions are dense in $L^1(\nu)$,

$$L^1(\nu) = \overline{\bigcup_{\pi \in \mathrm{Part}_{\mathrm{fin}}(\Omega)} \mathcal{C}_\pi}^{\,L^1(\nu)}.$$

Equivalently, one may regard $L^1(\nu)$ as the completion of the direct limit of the finite claim spaces.

This is the mathematical claim space toward which progressively finer partitions converge. An Arrow–Debreu complete market requires the additional economic condition that the relevant claims in this space are attainable or replicable.

2.3 Branchmap and controlled state-space refinement

Section 2.2.3 shows that every finite measurable partition defines a space of state-contingent claims. The contribution of BRANCHMAP is a shared semantic system for selecting, naming, relating, and composing a tractable family of economically meaningful partitions.

Let

$$\mathfrak{P}_{\textsc{Branchmap}} \subseteq \mathrm{Part}_{\mathrm{fin}}(\Omega)$$

denote the family of finite partitions induced by distinctions registered in BRANCHMAP. This family is expected to be a very small subset of all mathematically possible partitions of Ω.

BRANCHMAP does not define one global partition of the future. Different markets and applications may distinguish different aspects of the same world-state space and therefore operate with different, overlapping partitions. Each partition functions as a finite coordinate system for the state-contingent claims relevant to a particular financial problem.

These coordinate systems may be combined through common refinement. Given

$$\pi = \{V_1, \ldots, V_n\}, \qquad \rho = \{W_1, \ldots, W_m\},$$

their join is

$$\pi \vee \rho := \{V_i \cap W_j : V_i \cap W_j \neq \varnothing\}.$$

The join retains every distinction made by either partition.

For example, if

$$\pi_A = \{A, A^c\} \qquad \text{and} \qquad \pi_B = \{B, B^c\},$$

then

$$\pi_A \vee \pi_B = \{A \cap B,\ A \cap B^c,\ A^c \cap B,\ A^c \cap B^c\},$$

after omitting empty cells. This refinement supports assets conditioned simultaneously on the distinctions represented by A and B.

The namespace gives these distinctions and their resulting verses stable, machine-readable identities. This permits an asset, prediction market, DEX, lending protocol, or other application to refer to the same verse without independently reconstructing its semantics.

It also makes refinement relationships operational. If $\pi' \succeq \pi$, applications can determine which fine verses $W \in \pi'$ lie beneath a coarse verse $V \in \pi$. The coherence identities (7) and (8) can therefore be used to aggregate fine-grained markets, decompose coarse claims, and detect inconsistent prices across related venues.

The value of BRANCHMAP is in providing common coordinates through which state-contingent claims can be discovered, compared, and composed across independently operated protocols.

The appropriate degree of refinement remains an economic and computational design problem. Too little refinement leaves important risks bundled together and unpriced;

DRAFT

too much refinement fragments liquidity and increases the complexity of portfolio management. We expect AI agents to increase the level of useful complexity that markets can support by searching, pricing, and managing larger families of state-contingent claims.

BRANCHMAP therefore constructs a controlled substructure of the Arrow–Debreu state space: a growing family of economically meaningful distinctions for which identity, settlement, liquidity, and composability can be made machine-readable.

3. The Multiverse Map

In this section we introduce our model of the *multiverse map* as introduced in David White's introductory post on multiverse finance [citation]. We proceed by giving the high-level ontology, followed by mathematical definitions from probability theory, and then their corresponding BRANCHMAP definitions in terms of the BRANCHMAP smart contract structure.

3.1 Event ontology

We wish to formalize and generalize the following ontology:

World. The set of all possible future states of the world.

Event context. A real-world occurrence or observation context.

Predicate. A test, claim, or property that can be evaluated against an event or world-state.

Outcome. The value or result of evaluating a predicate, usually YES/NO or one of several branches.

Verse. A world or financial context selected by one or more predicate outcomes.

Consider an example: whether or not the Fed hikes the interest rate in July 2027.

Example 3.1: A FOMC predicate

Event context	July 2027 FOMC decision.
Predicate	$r_{\text{after}} > r_{\text{before}}$
Outcome	YES: the Fed hiked. NO: the Fed did not hike.
Verse	The universe selected by $r_{\text{after}} > r_{\text{before}}$

Here r_{before} and r_{after} denote the Federal Funds target range immediately before and after the July 2027 FOMC decision.

3.2 Mathematical formalism

Here we give the mathematical formalism for the ontology of section 3.1, and how it applies to example 3.1.

Definition 1 (World-state space)**.** *Let Ω denote the space of possible future world-states. A point $\omega \in \Omega$ specifies a complete possible realization of the future.*

Definition 2 (Event context)**.** *An* event context *is a real-world occurrence, decision, observation, or process whose realization may vary across possible world-states. We write the collection of event contexts as $\mathcal{E}$.*

Each event context $e \in \mathcal{E}$ is associated with an observation map

$$X_e : \Omega \to \mathsf{S}_e,$$

where S_e is the space of possible realizations of e.

We use the term *event context* instead of *event* to avoid a collision with the term event typically used in probability theory to mean a measurable subset $E \subseteq \Omega$.

Let e be the July 2027 FOMC decision event of example 3.1. A world-state $\omega \in \Omega$ contains everything that happens in that possible future. The observation map $X_e : \Omega \to \mathsf{S}_e$ selects the information from ω relevant to the FOMC decision:

$$X_e(\omega) = (r_{\text{before}}, r_{\text{after}}),$$

where $\mathsf{S}_e = \mathbb{R}^2$ is the space of possible before-and-after target rates. For example, $\omega_1, \omega_2 \in \Omega$ may be two possible future world states such that

$$X_e(\omega_1) = (4.25\%, 4.50\%), \quad X_e(\omega_2) = (4.25\%, 4.25\%).$$

Definition 3 (Predicate)**.** *A predicate on an event context e is a machine-readable test*

$$\widetilde{P} : \mathsf{S}_e \to \mathcal{O}_P,$$

where $\mathcal{O}_P$ is its outcome set. It induces a predicate on world-states

$$P := \widetilde{P} \circ X_e : \Omega \to \mathcal{O}_P.$$

When the event context is clear, we write simply

$$P : \Omega \to \mathcal{O}_P.$$

The predicate for "the Fed hikes" would be given by $\widetilde{P}_{\text{hike}} : \mathsf{S}_e \to \{\mathsf{YES}, \mathsf{NO}\}$ defined by

$$\widetilde{P}_{\text{hike}}(r_{\text{before}}, r_{\text{after}}) = \begin{cases} \mathsf{YES}, & r_{\text{after}} > r_{\text{before}}, \\ \mathsf{NO}, & r_{\text{after}} \le r_{\text{before}}. \end{cases}$$

Definition 4 (Outcome)**.** *An* outcome *is the value obtained by evaluating a predicate. If $P : \Omega \to \mathcal{O}_P$, then for a world-state $\omega \in \Omega$,*

$$o = P(\omega) \in \mathcal{O}_P$$

is an outcome of P. In the binary case, $\mathcal{O}_P = \{\mathsf{YES}, \mathsf{NO}\}$.

DRAFT

For our example, $P_{\text{hike}} := \widetilde{P}_{\text{hike}} \circ X_e : \Omega \to \{\mathsf{YES}, \mathsf{NO}\}$, so $P_{\text{hike}}(\omega_1) = \mathsf{YES}$, $P_{\text{hike}}(\omega_2) = \mathsf{NO}$.

Definition 5 (Verse)**.** *A* verse *is the financial world selected by one or more predicate outcomes. For a predicate P and outcome $o \in \mathcal{O}_P$, the corresponding verse is*

$$V_{P=o} := \{\omega \in \Omega : P(\omega) = o\}.$$

In other words, it is the inverse image $P^{-1}(o)$.

For multiple predicates $P_1, \ldots, P_n$ with selected outcomes $o_1, \ldots, o_n$, the induced verse is

$$V_{P_1=o_1,\ldots,P_n=o_n} := \{\omega \in \Omega : P_i(\omega) = o_i \text{ for all } i = 1, \ldots, n\}$$

The corresponding YES verse $V_{\text{hike}=\mathsf{YES}}$ in our example is

$$V_{\text{hike}=\mathsf{YES}} = \{\omega \in \Omega : P_{\text{hike}}(\omega) = \mathsf{YES}\}.$$

3.3 Branchmap realization

We now describe how the ontology of Section 3.2 is instantiated in BRANCHMAP. The protocol represents events, predicates, and outcomes as named onchain objects, while verses and conditional assets are derived canonically from combinations of predicate outcomes.

We separate the system into data and control planes.

Branchmap. The namespace and semantic registry. It gives events, predicates, and outcomes stable identifiers, ownership, metadata, provenance, and programmable token-bound accounts.

Controller. The financial controller and root owner of the smart contract suite. It validates predicates and outcomes, constructs canonical verses, manages resolution, holds root collateral, and creates conditional ERC-20 assets.

The existence of a controller with root access is a first stepping stone towards a more decentralized event namespace. For the purposes of this paper, we set aside discussing any details on how the namespace will be decentralized, classifying it as premature optimization.

3.3.1 Key-value store

As mentioned in subsection 1.4.2, BRANCHMAP is a fork of Hypermap [citation], which itself was inspired by dmap [citation], a minimalist path-formatted key-value store. BRANCHMAP is thus organized as a hierarchical set of paths much like a file system, and under each path some number of mutable and immutable keys and their corresponding values may be stored. These keys and values ultimately act as an index for the event space $\mathcal{E}$. Here we paraphrase a relevant section of the Hypermap white paper, as BRANCHMAP is identical:

1. All keys are strings containing exclusively characters `0-9`, `a-z` (lowercase), and `-` (hyphen).
2. A key may be one of two types: a *name-key* or a *data-key*.
3. Every name-key may create sub-entries directly beneath it.
4. Every name-key is an ERC-721 NFT, with an associated token-bound account having a counterfactual address, implemented according to the ERC-6551 standard.
5. The implementation of a token-bound account may be selected when a name-key is created.
6. Every name-key may create data-keys directly beneath it.
7. A data-key may be either mutable (a *note*, whose label is prefixed by `~`) or immutable (a *fact*, whose label is prefixed by `!`).

The root entry of BRANCHMAP is denoted `.branch`. We write entries in the namespace child-first, separated by dots, e.g. `grandchild.child.parent`.

3.3.2 Event-context entries

An event context $e \in \mathcal{E}$ is represented by a name-key directly beneath the `.branch` root. For example, the July 2027 FOMC decision may be represented by

`fomc-july-2027.branch`.

This entry identifies the shared real-world context to which one or more predicates may refer. It does not itself assert a proposition or select an outcome.

As a Branchmap name-key, the event-context entry is an ERC-721 token with an associated ERC-6551 token-bound account. Ownership of the entry therefore determines who may create or administer descendants beneath it, subject to any inherited account implementation or other namespace policy.

The entry may also carry data-keys describing the event context. Immutable facts may commit to properties such as its object type, semantic identifier, or parent series, while mutable notes may contain display metadata such as its title, description, scheduled time, jurisdiction, and metadata URI.

A minimal event-context subtree may therefore appear schematically as

```
fomc-july-2027.branch
|-- !kind
|-- !event-context-id
|-- ~title
|-- ~description
|-- ~scheduled-time
‘-- ~metadata-uri
```

The distinction between the name-key and its data-keys is important: the name-key is the ownable and programmable namespace object, whereas facts and notes describe that object.

DRAFT

3.3.3 Predicate entries

A predicate

$$P : \Omega \longrightarrow \mathcal{O}_P$$

defined on an event context is represented by a child name-key beneath the corresponding event-context entry.

For the predicate

$$P_{\text{hike}}(\omega) = \begin{cases} \mathsf{YES}, & r_{\text{after}}(\omega) > r_{\text{before}}(\omega), \\ \mathsf{NO}, & \text{otherwise}, \end{cases}$$

a corresponding namespace entry is

`target-range-inc.fomc-july-2027.branch`.

Its position beneath the event-context entry records that the predicate is defined with respect to the July 2027 FOMC decision. Other predicates concerning the same event context are represented by sibling entries, for example

`change-bps.fomc-july-2027.branch`,
`upper-bound-after.fomc-july-2027.branch`.

The predicate entry may carry immutable facts committing to its parent event context, outcome count, resolver, and semantic or settlement specification. Mutable notes may provide a human-readable title, description, or metadata URI.

Schematically,

```
target-range-inc.fomc-july-2027.branch
|-- !kind
|-- !predicate-id
|-- !event-context-id
|-- !outcome-count
|-- !resolver
|-- !specification-hash
|-- ~title
|-- ~description
‘-- ~metadata-uri
```

The full semantic specification of a predicate may include the measured quantity, comparison rule, observation window, resolution source, deadline, exception rules, and invalidation behavior. Two venue-specific predicates may therefore occupy distinct namespace entries even when they share a common event context and similar display text.

3.3.4 Outcome entries

For every $o \in \mathcal{O}_P$, the predicate entry has a child name-key representing the proposition

$$P(\omega) = o.$$

For the binary FOMC predicate, the resulting namespace is

```
branch
‘-- fomc-july-2027
    ‘-- target-range-inc
        |-- yes
        ‘-- no
```

The outcome entries are

`yes.target-range-inc.fomc-july-2027.branch`,
`no.target-range-inc.fomc-july-2027.branch`.

An outcome entry may carry immutable facts identifying its parent predicate, selected outcome value, and outcome index:

```
yes.target-range-inc.fomc-july-2027.branch
|-- !kind
|-- !outcome-id
|-- !predicate-id
|-- !outcome-index
|-- !outcome-value
|-- ~title
‘-- ~metadata-uri
```

The outcome name-key is the namespace object used by the financial layer when constructing verses. Its namehash, defined below in subsection 3.3.5, is the globally unique identifier of the selected predicate outcome.

3.3.5 Namehash derivation

Branchmap identifiers are derived recursively from the namespace hierarchy. Let

$$h_{\text{branch}} := \text{leaf}\big(0, \texttt{branch}\big)$$

be the identifier of the `.branch` root.

For the example above, define

$$h_e := \text{leaf}\big(h_{\text{branch}}, \texttt{fomc-july-2027}\big),$$

$$h_P := \text{leaf}\big(h_e, \texttt{target-range-inc}\big),$$

$$h_{\mathsf{YES}} := \text{leaf}\big(h_P, \texttt{yes}\big),$$

$$h_{\mathsf{NO}} := \text{leaf}\big(h_P, \texttt{no}\big).$$

We identify these hashes with the corresponding semantic identifiers:

$$\begin{aligned} \text{eventContextId}(e) &:= h_e, \\ \text{predicateId}(P) &:= h_P, \\ \text{outcomeId}(P, \mathsf{YES}) &:= h_{\mathsf{YES}}, \\ \text{outcomeId}(P, \mathsf{NO}) &:= h_{\mathsf{NO}}. \end{aligned}$$

Because each identifier is derived from its parent, an outcome identifier commits to the complete namespace path containing its outcome label, predicate, event context, and namespace root. The labels `yes` and `no` may be reused beneath different predicates without creating an identity collision.

3.3.6 Induced partitions

The predicate

$$P : \Omega \longrightarrow \mathcal{O}_P$$

induces the partition

$$\pi_P := \left\{ P^{-1}(o) \subseteq \Omega : o \in \mathcal{O}_P \right\}.$$

DRAFT

Each outcome entry labels one cell $P^{-1}(o)$ of this partition. The partition is therefore represented in the namespace by a predicate entry together with its complete set of outcome children:

```
predicate
|-- outcome-1
|-- outcome-2
| ...
‘-- outcome-n
```

No additional partition name-key is required. The predicate entry identifies the outcome-valued map, and its child entries identify the cells of the partition induced by that map.

If a distinct partition identifier is useful for an external interface, it may simply be identified with predicateId(P), since the predicate and its complete outcome set determine π_P.

3.3.7 Verses

Unlike event contexts, predicates, and outcomes, *verses* are not primitive namespace entries. A verse is a canonical object derived from a compatible set of outcome-entry identifiers. Verses are ultimately the datum used to identify conditional assets, in-universe defi protocols, etc.

Let

$$S = \{(P_1, o_1), \ldots, (P_n, o_n)\}$$

be a finite set of selected predicate outcomes. Its corresponding verse is

$$V_S := \bigcap_{(P,o)\in S} P^{-1}(o).$$

Within the contracts, this verse is represented by the set

$$\widehat{S} = \{\text{outcomeId}(P_1, o_1), \ldots, \text{outcomeId}(P_n, o_n)\}.$$

A valid verse may contain at most one outcome associated with any given predicate. Meaning,

$$(P, o_1), (P, o_2) \in S \quad \Longrightarrow \quad o_1 = o_2.$$

Let d_{verse} denote a hash domain-separation constant `BRANCHPOINT_VERSE_V1`, and define

$$d_{\text{verse}} := \text{keccak256}\big(\texttt{BRANCHPOINT_VERSE_V1}\big),$$
$$H_{\text{verse}}(x) := \text{keccak256}\big(\text{abi.encode}(d_{\text{verse}}, x)\big).$$

The canonical verse identifier is then

$$\text{verseId}(S) := H_{\text{verse}}\big(\text{sort}(\widehat{S})\big). \tag{9}$$

The unconditional or *top verse* $V_\top$ corresponds to the empty set of predicate–outcome assignments,

$$S_\top := \varnothing.$$

Since no conditions have been imposed, this verse contains every possible world-state:

$$V_\top := V_\varnothing = \bigcap_{(P,o)\in\varnothing} P^{-1}(o) = \Omega.$$

Here we use the standard convention that the intersection of an empty family of subsets of Ω is Ω itself.

At the contract level, the top verse is assigned the distinguished identifier

$$\texttt{TOP_VERSE_ID} := \texttt{bytes32}(0).$$

It is a reserved identifier for the empty outcome set, not a probability, outcome value, or hash of a namespace entry.

The presence of sorting in equation (9) implies that the ordering of predicates is immaterial. An asset conditioned first on the FOMC decision and then on a presidential election must be fungible with an asset conditioned on the same outcomes in the reverse order. The Gnosis CTF possesses the same property.

As predicate order-independence is crucial, verses cannot be identified by nested namespace paths therefore cannot be a primitive. The BRANCHMAP namespace names the primitive events, predicates, and outcomes, while the controller derives canonical verse IDs from unordered sets of outcome identifiers. Human-readable verse entries may later be materialized as aliases or “world pages,” but they do not determine financial identity.

3.3.8 Conditional assets

Let R be a root ERC-20 asset and let S be a valid verse assignment set. We write the asset R conditioned on S as

$$R@S$$

or, when the intended meaning is clear,

$$R@S,$$

reminiscent of conditional probability theory notation.

Its canonical financial identity is the pair

$$(R, \text{verseId}(S)).$$

For every such pair, the controller creates at most one conditional ERC-20 address. Consequently,

$$\mathsf{BTC}@\{P_{\text{hike}} = \mathsf{YES}, P_{\text{btc100k}} = \mathsf{YES}\}$$

denotes one canonical BTC asset inside the verse where both selected outcomes occur.

The same verse may contain many different root assets:

$$\mathsf{USDC}@S, \qquad \mathsf{BTC}@S, \qquad \mathsf{ETH}@S.$$

Because these assets share the same verseId(S), external protocols can determine that they inhabit the same financial world. This provides the basis for verse-aware AMMs, lending markets, vaults, and structured products.

An ordinary ERC-20 asset such as BTC or USDC is treated as inhabiting the top verse $V_\top$ because it is not yet conditioned on any event outcome. For example,

$$\mathsf{BTC} \equiv \mathsf{BTC}@\varnothing.$$

DRAFT

3.3.9 Splitting and recombining

Suppose an asset $R@S$ is split through a predicate

$$P : \Omega \longrightarrow \{o_1, \ldots, o_m\}.$$

Provided that S does not already contain an assignment for P, the split produces

$$R@\left(S \cup \{P = o_1\}\right), \ldots, R@\left(S \cup \{P = o_m\}\right).$$

Each output has the same quantity as the input, but is valid only in its corresponding branch. For a binary predicate,

$$R@S \longrightarrow \left(R@\left(S \cup \{P = \mathsf{YES}\}\right), R@\left(S \cup \{P = \mathsf{NO}\}\right)\right).$$

A root asset R is treated as inhabiting the top verse $V_\top$. Repeated splitting therefore constructs deeper conditional assets while retaining a canonical, order-independent verse identity.

Splitting R through a binary predicate P replaces this empty assignment set with one of two singleton sets:

$$R@\varnothing \longrightarrow \left(R@\{P = \mathsf{YES}\}, R@\{P = \mathsf{NO}\}\right).$$

Conversely, as selected outcomes resolve successfully, they are removed from a conditional asset's effective verse. If every assignment is removed, the asset returns to the empty outcome set and becomes redeemable for the ordinary root asset:

$$R@\{P_1 = o_1, \ldots, P_n = o_n\} \longrightarrow R@\varnothing \equiv R.$$

3.3.10 Resolution and normalization

A predicate resolves when its authorized resolution mechanism, typically an oracle, reports a value

$$o_P^* \in \mathcal{O}_P.$$

Let

$$S = \{(P_1, o_1), \ldots, (P_n, o_n)\}$$

be the verse associated with a conditional asset. Resolution normalizes S according to the following rules:

1. If P_i is unresolved, the assignment (P_i, o_i) remains.
2. If P_i resolves to o_i, the assignment is satisfied and is removed.
3. If P_i resolves to some $o_i^* \neq o_i$, the verse is inconsistent with the realized world and the position is invalidated.

We write the normalized verse, meaning the verse that the asset occupies after resolution, as

$$N(S) = \{(P_i, o_i) \in S : P_i \text{ remains unresolved}\},$$

provided that no resolved predicate contradicts its selected outcome.

For example,

$$R@\{P_A = \mathsf{YES}, P_B = \mathsf{YES}\} \xrightarrow[P_B=\mathsf{YES}]{P_B \text{ resolves}} R@\{P_A = \mathsf{YES}\},$$

whereas

$$R@\{P_A = \mathsf{YES}, P_B = \mathsf{YES}\} \xrightarrow[P_B=\mathsf{NO}]{P_B \text{ resolves}} 0.$$

If every selected outcome resolves as specified, normalization reaches the top verse and the conditional asset becomes redeemable for its root asset:

$$R@S \longrightarrow R.$$

This normalization gives the multiverse map a dynamic interpretation. As uncertainty is resolved, inconsistent verses disappear, while outcomes consistent with reality are stripped from surviving verses until unconditional assets remain.

3.3.11 Namespace ownership and provenance

Every event, predicate, and outcome entry is an ERC-721 namespace object with an associated ERC-6551 token-bound account. These accounts provide a programmable locus for ownership, delegation, metadata, and venue-specific behavior.

During the pilot, BRANCHPOINT may create entries from public venue data and mark them as observed but unclaimed. Event venues may later claim and control their own namespace subspaces without changing the identifiers or historical provenance of previously registered objects.

BRANCHMAP distinguishes immutable semantic commitments from mutable presentation metadata. Immutable facts may include

- the object kind,
- event, predicate, and outcome identifiers,
- references between predicates and events,
- outcome counts and indices,
- resolver addresses,
- semantic or settlement-specification hashes, and
- protocol-version identifiers.

Mutable notes may include titles, descriptions, display labels, images, and metadata URIs. In this way, BRANCHMAP separates what the protocol commits to, what an authorized venue asserts, and how applications choose to present the resulting multiverse map.

4. Applications of Branchmap

In this section we discuss possible uses for BRANCHMAP. The design space for this is extremely large, and we expect many applications that the authors do not foresee.

DRAFT

4.1 Impact markets

Prediction markets price whether an event occurs. *Impact markets* instead price how the occurrence of that event changes the value of an asset.

Let P be a binary event with corresponding verses

$$V_{\mathsf{YES}} := P^{-1}(\mathsf{YES}), \qquad V_{\mathsf{NO}} := P^{-1}(\mathsf{NO}).$$

Let

$$p := \mathbb{Q}(V_{\mathsf{YES}}) \in (0,1)$$

be the market-implied pricing probability of the YES verse, so that

$$\mathbb{Q}(V_{\mathsf{NO}}) = 1 - p.$$

Suppose an asset X can be decomposed into complementary conditional claims,

$$X \longleftrightarrow X \mid V_{\mathsf{YES}} + X \mid V_{\mathsf{NO}}.$$

Write their branch prices as

$$C_X^{\mathsf{YES}} := C_X(V_{\mathsf{YES}}), \qquad C_X^{\mathsf{NO}} := C_X(V_{\mathsf{NO}}).$$

Define

$$q := \frac{C_X^{\mathsf{YES}}}{C_X^{\mathsf{YES}} + C_X^{\mathsf{NO}}} \in (0,1),$$

the fraction of the current value of X assigned by the market to the YES-contingent claim. Equivalently, if

$$\rho_{X,P} := \frac{C_X^{\mathsf{YES}}}{C_X^{\mathsf{NO}}},$$

then

$$q = \frac{\rho_{X,P}}{1 + \rho_{X,P}}.$$

The quantity q combines two distinct effects: the state price of each verse and the value of X inside that verse. Applying the branch price factorization of equation (3) gives

$$C_X^{\mathsf{YES}} = D_{0T}\, p\, m_X(V_{\mathsf{YES}}),$$
$$C_X^{\mathsf{NO}} = D_{0T}\, (1-p)\, m_X(V_{\mathsf{NO}}).$$

Consequently,

$$\frac{q}{1-q} = \frac{p}{1-p} \frac{m_X(V_{\mathsf{YES}})}{m_X(V_{\mathsf{NO}})}.$$

Branchpoint defines the *impact* of P on X by

$$I_{X,P} := \operatorname{logit}(q) - \operatorname{logit}(p), \tag{10}$$

where $\operatorname{logit}(x) := \log\left(\frac{x}{1-x}\right)$. For positive in-verse asset values, the preceding factorization implies the equivalent expression

$$I_{X,P} = \log\left(\frac{m_X(V_{\mathsf{YES}})}{m_X(V_{\mathsf{NO}})}\right). \tag{11}$$

Thus, the impact statistic is the logarithmic ratio between the market-implied value of X in the YES verse and its market-implied value in the NO verse. In particular,

$$\begin{aligned} I_{X,P} = 0 &\iff m_X(V_{\mathsf{YES}}) = m_X(V_{\mathsf{NO}}), \\ I_{X,P} > 0 &\iff m_X(V_{\mathsf{YES}}) > m_X(V_{\mathsf{NO}}), \\ I_{X,P} < 0 &\iff m_X(V_{\mathsf{YES}}) < m_X(V_{\mathsf{NO}}). \end{aligned}$$

Prediction markets therefore determine the relative state prices of the two verses, while conditional-asset markets determine the value of the asset inside each verse. BRANCHMAP supplies the shared semantic identities required to ensure that both sets of prices refer to the same event, outcomes, and settlement rules.

Equation (11) measures a market-implied conditional valuation difference. It should be given a causal interpretation only when the market design and identifying assumptions justify treating P as an intervention rather than merely a correlated event.

4.2 Options as a special case of multiverse finance

Traditional options may be understood as a special case of multiverse finance in which the relevant branches are defined by the future price of an underlying asset.

Let X be an asset with terminal price S_T at time T. For a strike K, define the binary predicate

$$P_{K,T} : \Omega \longrightarrow \{\mathsf{YES}, \mathsf{NO}\}$$

by

$$P_{K,T}(\omega) = \begin{cases} \mathsf{YES}, & S_T(\omega) > K, \\ \mathsf{NO}, & S_T(\omega) \le K. \end{cases}$$

This predicate partitions the possible future world-states into the two verses

$$V_{K,T}^{+} := \{\omega \in \Omega : S_T(\omega) > K\},$$
$$V_{K,T}^{-} := \{\omega \in \Omega : S_T(\omega) \le K\}.$$

Let M denote one unit of the cash or settlement asset. A binary cash-or-nothing call is precisely conditional cash or "verse cash" in the positive verse:

$$M \mid V_{K,T}^{+}.$$

Equivalently, verse cash is a standard prediction market contract that resolves to \$1 in the positive verse. Likewise, an asset-or-nothing call is the conditional underlying

$$X \mid V_{K,T}^{+}.$$

A standard European call option is the portfolio

$$X \mid V_{K,T}^{+} - K\big(M \mid V_{K,T}^{+}\big).$$

At maturity, this portfolio pays

$$S_T - K$$

DRAFT

in the verse where $S_T > K$, and zero otherwise. Its payoff is therefore

$$\big(S_T - K\big)^+.$$

Similarly, a European put may be written as

$$K\big(M \mid V^-_{K,T}\big) - X \mid V^-_{K,T},$$

which pays

$$\big(K - S_T\big)^+.$$

From this perspective, an option is a portfolio of assets living in different branches of a partition of the future state space. The option contract packages those branch-specific positions into a single instrument and reports only their net terminal payoff.

More generally, suppose a predicate

$$P : \Omega \longrightarrow \mathcal{O}_P$$

induces a finite partition of Ω, and let Y_o be the payoff specified in outcome o for $o \in \mathcal{O}_P$. The corresponding contingent claim may be written

$$Y_T = \sum_{o \in \mathcal{O}_P} Y_o \mathbf{1}_{\{P=o\}}.$$

Multiverse finance makes the summands $Y_o\mathbf{1}_{\{P=o\}}$ explicitly identifiable, ownable, and composable. Vanilla options are the elementary two-branch case in which the predicate concerns the terminal price of the underlying and one branch has zero payoff.

An option chain may likewise be viewed as a family of related predicates indexed by strike and maturity. For fixed T, the predicates are nested:

$$K_1 < K_2 \quad \Longrightarrow \quad V^+_{K_2,T} \subseteq V^+_{K_1,T}.$$

A semantic namespace can record these implication and refinement relationships rather than treating every strike as an unrelated contract.

Multiverse finance can therefore be thought of as a broad generalization of options. Traditional options collapse conditional worlds into a single payoff, while multiverse finance preserves the conditional assets themselves, allowing assets, liabilities, markets, vaults, and contracts to continue operating inside the selected verse.

Moreover, the predicates need not concern the future price of the underlying itself. They may refer to elections, policy decisions, protocol upgrades, scientific results, personal milestones, or combinations of several events. In this sense, options are the familiar price-contingent corner of a much larger design space of world-contingent finance.

This perspective would allow, for instance, cryptocurrency options to be constructed utilizing the existing popular cryptocurrency up/down prediction market contracts.

4.3 Multiverse-only instruments

BRANCHMAP allows for instruments that are not merely existing financial instruments with a conditional wrapper. It allows for completely new instruments native to multiverse finance that only make sense when future states are first-class financial contexts.

4.3.1 Multiverse HELOC

A multiverse home-equity line of credit allows borrowing terms to vary across future states. For example, a user may borrow against the value of their home in a verse where the expected value of that home has risen, and then sell that claim to another party (likely a sophisticated one that specializes in pricing such claims). More generally, a person may borrow against future income, equity, or asset values only in worlds where specified macroeconomic or personal conditions hold. This allows for, among other things, a generalization of the concept of a collar in options trading. They can sell excess upside and use it to buy downside protection.

4.3.2 Conditional smart contracts

A conditional smart contract changes its execution path depending on event outcomes. Unlike an ordinary contract with an oracle trigger, a conditional smart contract can hold, route, or settle assets that already live inside the corresponding verse.

4.3.3 Cybernetic hypothesis vault

A cybernetic hypothesis vault allocates capital across hypotheses, observes outcomes, and updates future allocations based on measured causal impact. Such a vault requires named hypotheses, interventions, outcome claims, and impact measurements.

4.3.4 Counterfactual protocol rollout

A protocol can issue assets or incentives conditional on whether a proposed rollout, parameter change, or governance action succeeds. This allows communities to trade and fund counterfactual protocol futures before committing to them.

4.3.5 Startup funding

Startup financing can become conditional on milestones, market states, governance events, product launches, or ecosystem adoption. Multiverse-native instruments can encode upside, downside, and contingent support more precisely than conventional SAFEs, notes, or equity.

4.4 Relationship to Futarchy

Futarchy proposes governance by betting on outcomes conditional on policy decisions. Multiverse finance generalizes this idea. Instead of only asking which policy maximizes a scalar welfare metric, it allows entire financial systems to be instantiated, traded, and compared across policy-conditioned worlds.

DRAFT

5. Mathematics of a multiverse portfolio

A multiverse portfolio is a collection of conditional claims on assets indexed by verses. Let

$$\pi = \{V_1, \dots, V_n\}$$

be a finite partition of Ω into verses.

Let $\Phi_{i,T}$ denote the terminal payoff of the portfolio component active in verse V_i. The total terminal payoff is

$$\Phi_T = \sum_{i=1}^{n} \mathbf{1}_{V_i} \Phi_{i,T},$$

where

$$\mathbf{1}_{V_i}(\omega) = \begin{cases} 1, & \omega \in V_i, \\ 0, & \omega \notin V_i. \end{cases}$$

Exactly one indicator is nonzero in any terminal world-state, so the realized portfolio is the component belonging to the realized verse.

5.1 Verse probabilities from prediction markets

For each terminal verse V_i, consider a prediction market claim B_i that pays one unit of a settlement asset at time T if and only if V_i occurs:

$$B_i(T) = \mathbf{1}_{V_i}.$$

Let ψ_i denote its current market price. These prices are state prices: they measure the present value assigned by the market to one unit of payoff in each verse.

Because the verses are mutually exclusive and exhaustive, a complete set of these claims pays one unit in every possible world:

$$\sum_{i=1}^{n} B_i(T) = 1.$$

The fact that assets may be split and recombined across verses (see section 3.3.9) means that no-arbitrage implies that their prices must satisfy

$$\sum_{i=1}^{n} \psi_i = D_{0T},$$

where D_{0T} is the current price of one unit of the settlement asset delivered at time T.

The corresponding market-implied probability of verse V_i is

$$q_i := \frac{\psi_i}{D_{0T}}.$$

It follows that

$$q_i \geq 0, \qquad \sum_{i=1}^{n} q_i = 1.$$

When the prediction-market collateral is non-yielding and one complete set may be split from or recombined into one unit of collateral, D_{0T} is effectively one, and the outcome-token prices themselves may be read directly as probabilities:

$$q_i \approx \psi_i.$$

These are market-implied pricing probabilities. They need not equal the true physical frequencies of the events or any particular participant's subjective beliefs.

5.2 Expected portfolio value

Let $\mathbb{Q}$ denote the pricing measure on Ω implied by the prediction market, so that

$$\mathbb{Q}(V_i) = q_i.$$

The market-implied expectation of the multiverse portfolio is then

$$\begin{aligned} \mathbb{E}^{\mathbb{Q}}\big[\Phi_T\big] &= \mathbb{E}^{\mathbb{Q}}\left[\sum_{i=1}^{n} \mathbf{1}_{V_i} \Phi_{i,T}\right] \\ &= \sum_{i=1}^{n} q_i \, \mathbb{E}^{\mathbb{Q}}\left[\Phi_{i,T} \mid V_i\right]. \end{aligned}$$

Thus, the expected value is obtained by weighting the value of the portfolio in each verse by the probability assigned to that verse by prediction markets.

Discounting to the present gives

$$V_0(\Phi) = D_{0T} \sum_{i=1}^{n} q_i \, \mathbb{E}^{\mathbb{Q}}\left[\Phi_{i,T} \mid V_i\right].$$

Equivalently, using the prediction-market state prices directly,

$$V_0(\Phi) = \sum_{i=1}^{n} \psi_i \, \mathbb{E}^{\mathbb{Q}}\left[\Phi_{i,T} \mid V_i\right].$$

If each verse-specific payoff is deterministic, this simplifies to

$$V_0(\Phi) = D_{0T} \sum_{i=1}^{n} q_i \, \Phi_{i,T}.$$

For a binary predicate P, with

$$p := \mathbb{Q}(P = \mathsf{YES}),$$

a portfolio with payoffs $\Phi_{\mathsf{YES},T}$ and $\Phi_{\mathsf{NO},T}$ has expected terminal value

$$\begin{aligned} \mathbb{E}^{\mathbb{Q}}[\Phi_T] = p\,\mathbb{E}^{\mathbb{Q}}[\Phi_{\mathsf{YES},T} \mid P = \mathsf{YES}] \\ + (1-p)\,\mathbb{E}^{\mathbb{Q}}[\Phi_{\mathsf{NO},T} \mid P = \mathsf{NO}]. \end{aligned}$$

5.2.1 Conditional asset prices

Prediction markets determine how much weight the market assigns to each verse. Markets for conditional assets determine what an asset is worth inside each verse.

Let $X \mid V_i$ be a claim that delivers the terminal value X_T if V_i occurs and zero otherwise. Its present price, denoted $C_X(V_i)$, is

$$C_X(V_i) = D_{0T}\, q_i \, \mathbb{E}^{\mathbb{Q}}\left[X_T \mid V_i\right].$$

DRAFT

Since

$$\psi_i = D_{0T} q_i,$$

the market-implied conditional value of X inside the verse may be recovered as

$$\mathbb{E}^{\mathbb{Q}}[X_T \mid V_i] = \frac{C_X(V_i)}{\psi_i},$$

provided that $\psi_i > 0$.

This separates two quantities that are combined in the raw price of a conditional asset:

1. the probability that the verse occurs, supplied by the prediction market; and
2. the value of the asset conditional on that verse occurring, supplied by the conditional-asset market.

If X is split across a complete terminal partition,

$$X \longleftrightarrow \sum_{i=1}^{n} X \mid V_i,$$

then no arbitrage implies

$$V_0(X) = \sum_{i=1}^{n} C_X(V_i).$$

The ordinary spot asset is therefore the aggregate of its verse-contingent components.

5.2.2 Nested verses

For verses defined by several predicates, the relevant weights are joint probabilities. For example,

$$q_{A=\mathsf{YES},B=\mathsf{YES}} = \mathbb{Q}\big(A = \mathsf{YES},\, B = \mathsf{YES}\big).$$

In general,

$$q_{A=\mathsf{YES},B=\mathsf{YES}} \neq q_{A=\mathsf{YES}}\, q_{B=\mathsf{YES}},$$

unless the two predicates are independent under the market-implied measure. Marginal prediction-market prices alone therefore do not determine the value of a multi-event verse. Its probability must be obtained from a market for the joint claim, from a coherent conditional market, or from an explicit model of dependence.

BRANCHMAP provides the common identities needed to perform these calculations safely. It ensures that the prediction-market state price, the conditional asset, and the portfolio component all refer to the same predicate outcomes and settlement specifications. In this way, prediction markets provide the probability distribution over the multiverse, while multiverse financial markets determine the allocation and value of assets within each branch.

6. Multiverse Plumbing

6.1 Integration Model

BRANCHMAP does not require venues to adopt a specific exchange architecture. A venue can integrate by publishing or claiming:

1. event identifiers;
2. predicate identifiers;
3. settlement specifications;
4. oracle references;
5. market contracts;
6. conditional-token identifiers;
7. metadata hashes;
8. lifecycle status;
9. resolution data.

6.2 Market Manifests

A venue can describe a market using a signed manifest:

```
MarketManifest {
    chainId;
    venueId;
    sourceMarketId;
    eventContextRef;
    predicateRef;
    settlementSpecRef;
    collateral;
    conditionalTokenContract;
    conditionId;
    outcomeTokenIds;
    oracle;
    metadataHash;
    revision;
}
```

Initially, manifests may be inferred and signed by BRANCHPOINT. Over time, venues should publish their own signed manifests.

6.3 Single-Chain Pilot

The first BRANCHMAP deployment is expected to be a single-chain EVM namespace. It may be governed by a BRANCHPOINT multisig during the pilot period. Venues and protocols may nevertheless control their own subspaces through delegated ownership, permissioned writers, or claim mechanisms.

6.4 Toward a Cross-Chain Namespace

The initial EVM namespace is a pilot program for a future cross-chain namespace. In the cross-chain design, semantic objects such as events, predicates, and settlement specifications should be chain-independent, while

DRAFT

instruments, wrappers, vaults, and settlement transactions remain chain-specific.

Event	:	chain-independent
Proposition	:	chain-independent
Settlement Specification	:	venue- or oracle-specific
Conditional Asset	:	chain-specific
Branch Cash Wrapper	:	chain-specific
Venue Market	:	venue- and chain-specific

6.5 Governance Path

The governance path is expected to progress through stages:

1. **Pilot curation:** BRANCHPOINT multisig populates and maintains the initial namespace.
2. **Venue claims:** event venues and DEXes claim their own subspaces.
3. **Signed publishing:** venues publish signed manifests directly.
4. **Curator networks:** independent curators assert semantic relationships.
5. **Decentralized governance:** a future token or governance system manages root policy, disputes, fees, and upgrades.

7. Application integration

One of the main ideas behind BRANCHMAP is to make it easy for existing protocols to integrate multiverse finance. Here we discuss what this may look like.

7.1 DEX integration

A DEX can integrate with BRANCHMAP by recognizing Branch Cash wrappers, reading verse metadata, and enforcing same-verse composition rules where appropriate. The DEX need not adopt any particular AMM design.

7.2 Prediction market integration

A prediction market can integrate by claiming a venue subspace and publishing signed market manifests. This allows its markets to become discoverable, comparable, and composable across the broader multiverse finance ecosystem.

7.3 Lending protocol integration

A lending protocol can use BRANCHMAP to determine which conditional assets inhabit compatible verses. This enables lending markets where collateral and debt share the same conditional state.

7.4 Structured products

Structured products can be represented as portfolios over verses. A product may pay differently across election outcomes, macroeconomic regimes, governance decisions, protocol launches, or personal milestones.

7.5 AI Agents

AI agents require machine-readable market structure, event identity, settlement rules, and cross-venue relationships. BRANCHMAP provides the semantic substrate for agents to discover, compare, and compose multiverse financial opportunities.

8. Security and Risk Considerations

To be written

9. Conclusion

BRANCHMAP is a namespace for the financial multiverse that gives a concrete implementation path towards Arrow-Debreu complete markets. Conditional-token primitives make it possible to create assets whose payoffs depend on future outcomes, but they do not by themselves create a navigable, interoperable, or institutionally credible financial system. BRANCHMAP supplies the missing map: names, relationships, provenance, ownership, and integration surfaces for events, claims, venues, conditional assets, and in-universe protocols.

The initial EVM pilot aims to demonstrate this architecture on a single chain. The long-term goal is a decentralized cross-chain namespace where prediction markets, DEXes, lending protocols, structured products, AI agents, and event venues can coordinate around shared future-state identities without collapsing important settlement distinctions.

References

DRAFT